

↓ UNION (VARIANTE)

Esercizio 1 (9 punti) Realizzare una funzione `intersect(A1,A2,n)` che dati due array di interi `A1` e `A2`, organizzati a min-heap, con capacità `n`, restituisce un nuovo array `A`, ancora organizzato a min-heap, che contiene l'intersezione dei valori contenuti in `A1` e `A2`. Nel caso gli array contengano più occorrenze dello stesso valore `v`, l'intersezione mantiene il numero minimo di occorrenze di `v` (ad es. se `A1` contiene i valori 1,2,2,2 e `A2` contiene i valori 1,1,2,2 allora `A` conterrà 1,2,2). Valutarne la complessità.

↓ → EXTRACT MIN
↓ → MIN HEAPIFY ...

└──────────┘

PUOI USARE
FUNZIONE "NOTA VERA"!

EXTRACT-MAX(A) (MIN)
1 `max = A[1]` $\text{max} \rightarrow A[1]$
2 `A[1] = A[A.heapsize]`
3 `A.heapsize = A.heapsize - 1`
4 `MIN-HEAPIFY(A, 1)` // ripristina le proprietà di MinHeap
5 `return max`

```
intersect(A1, A2, n)
    allocate A (1..n)
    i = 0, j = 0

    while (a1.heapsize > 0 && a2.heapsize > 0)
        v1 = Min(A1)
        v2 = Min(A2)

        if (v1 == v2)
            ExtractMin(A1)
            ExtractMin(A2)
            i++, j++
        // se non uguali → vado avanti

        if (v1 < v2)
            ExtractMin(A1)
            i++
        else
            ExtractMin(A2)
            j++

    A.heapsize = i
    return A
```

```
intersect(A1,A2,n)
    allocate A[1..n]
    i=0
    while (A1.heapsize > 0) and (A2.heapsize > 0)
        v1 = Min(A1)
        v2 = Min(A2)
        if (v1 == v2)
            i++
            A[i]=1
            ExtractMin(A1)
            ExtractMin(A2)
        elseif (v1 < v2)
            ExtractMin(A1)
        else
            ExtractMin(A2)

    A.heapsize=i
    return A
```

Intersect: dati due min heap A1 e A2, restituisco un array che allochiamo A anche lui a min heap.

// ultimo elemento occupato in A

Finché esistono entrambi gli heap, essendo dei min-heap, allora estraiamo l'elemento minimo. Allora:

- se l'elemento è uguale tra i due heap, aumento l'indice, la radice di A (che parte da 1) e poi continuiamo a estrarre il minimo elemento
- se invece abbiamo l'elemento minore nel primo heap, lo estraiamo dal primo e lo mettiamo nell'array A
- se invece abbiamo l'elemento minore nel secondo heap, lo estraiamo dal secondo e lo mettiamo nell'array A
- la posizione di analisi di A è "i" e ritorniamo, a fine ciclo, A.

Essendo un heap, avremo che il costo è $n \log n$.

La complessità di ogni `ExtractMin` è $O(\log n)$ mentre `Min` ha costo costante. Il numero di estrazioni è chiaramente limitato da $2n$ e questo porta a dedurre che il costo complessivo è $O(n \log n)$.

STAB

Esercizio 1 (8 punti) Realizzare una funzione `missing(A,n)` che dato un array `A[1..n]` che contiene interi nell'intervallo `[1,n]` verifica se esiste qualche valore $v \in [1,n]$ che non compare in `A`, ovvero tale che $A[i] \neq v$ per ogni $i \in [1,n]$. Valutare la complessità.

COUNTING SORT $\rightarrow$ $B[A[i]]$ (IN LOCO / IN PLACE)
 $\downarrow$
 ASSIGNAZIONE

`missing(A, n)`

allocate `B(1..n)`

```
for i = 1 to n
  if (A[i] == v)
    B[i] == true
  (B[i] = false)
```

```
for i = 1 to n
  (if B[i] == false
   print("Value not found"))
```

```
missing(A,n)
  alloca B[1..n] array di booleani

  (for i=1 to n
   B[i]=false)

  i=1
  while (i<=n) and not B[A[i]]
    B[A[i]] = true
    i++

  return i<=n
```

$\rightarrow$ if $i \leq n \rightarrow$ TRUE
 $\rightarrow$ ASS $\rightarrow$ FALSE

$\Theta(m)$

Domanda A (7 punti) Si consideri la funzione ricorsiva `search(A,p,r,k)` che dato un array `A`, ordinato in modo decrescente, un valore `k` e due indici `p,q` con $1 \leq p \leq r \leq A.length$ restituisce un indice i tale che $p \leq i \leq r$ e $A[i] = k$, se esiste, e altrimenti restituisce 0. (Già esaminata a pag. 83)

```
search(A,p,r,k)
  if p <= r
    q = (p+r)/2
    if A[q] = k
      return q
    elseif A[q] > k
      return search(A,q+1,r,k)
    else
      return search(A,p,q-1,k)
  else
    return 0
```

$m/2$ $\rightarrow$ $m/2$ $\rightarrow$ $\log(m)$ $\rightarrow$ HALF $\rightarrow m/2$ $\rightarrow$ $2 \frac{m}{2} + c, \dots$
 $RSC(\dots) + RSC(\dots)$

Dimostrare induttivamente che la funzione è corretta. Inoltre, determinare la ricorrenza che esprime la complessità della funzione e risolverla con il Master Theorem.

$$T(m) = T\left(\frac{m}{2}\right) + c(1) = \Theta(\log(m))$$

(1 CHIAM. REC. II ALTRA CHIAM. REC.)...

$$m^{\log_b(a)} = m^{\log_2(1)} = m^0 = m$$

$$T(m) = \Theta(m \log_b m)$$

$$\log(m) = 1 \cdot \log(m) = \log(m)$$

Rispetto allo schema standard del master theorem ho $a = 1$, $b = 2$ e $f(n) = c$. Ho dunque che $f(n) = 1 = \Theta(n^{\log_2 1}) = \Theta(n^0) = \Theta(1)$. Pertanto si conclude che $T(n) = \Theta(n^0 \log n) = \Theta(\log n)$.

$$(q+1 \leq i \leq m) \rightarrow m/2 \rightarrow (i \leq q-1 \leq m) \rightarrow m/2$$

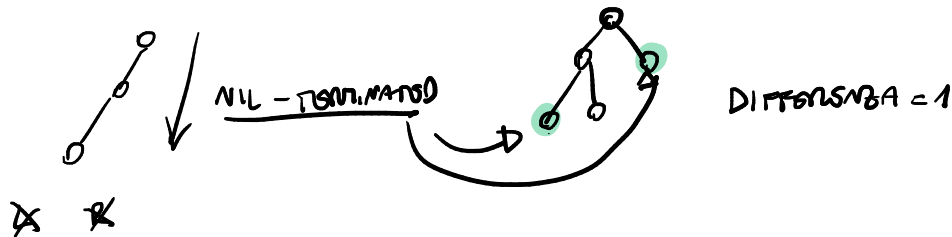
Soluzione: La prova di correttezza avviene per induzione sulla lunghezza $l = r - p + 1$ del sottoarray $A[p..r]$ di interesse. Se $l = 0$, ovvero $p > r$, la funzione ritorna correttamente 0, dato che non ci sono elementi nel sottoarray e quindi non ci possono essere elementi $= k$. Se invece $l > 0$ si calcola $q = \lfloor (p+r)/2 \rfloor$ e si distinguono tre casi:

- se $A[q] = k$ si ritorna correttamente k
- se $A[q] > k$, dato che l'array è ordinato in modo decrescente certamente $A[j] \geq A[q] > k$ per $p \leq j \leq q$. Quindi il valore k può trovarsi solo nel sottoarray $A[q+1, r]$. La chiamata $search(A, q+1, r, k)$, dato che il sottoarray ha lunghezza minore di l , per ipotesi induttiva restituisce un indice i tale che $q+1 \leq i \leq r$ e $A[i] = k$, se esiste, e altrimenti restituisce 0. Per l'osservazione precedente, questo è il valore corretto da restituire anche per $search(A, p, r, k)$ e si conclude.
- se $A[q] < k$ si ragiona in modo duale rispetto al caso precedente.

Per quanto riguarda la ricorrenza, ignorando i casi base, dato che la funzione ricorre su di un array la cui dimensione è la metà di quello originale, si ottiene:

$$T(n) = T(n/2) + c$$

Esercizio 1 (9 punti) Si consideri un albero binario T , i cui nodi x hanno i campi $x.l$, $x.r$, $x.p$ che rappresentano il figlio sinistro, il figlio destro e il padre, rispettivamente. Un *cammino* è una sequenza di nodi $x_0, x_1, \dots, x_n$ tale che per ogni $i = 0, \dots, n-1$ vale $x_{i+1}.p = x_i$. Il cammino è detto *nil-terminated* se $x_n.l = nil$ oppure $x_n.r = nil$. Dato un certo k , diciamo che l'albero è k -bilanciato se tutti i cammini nil-terminated che iniziano dalla radice hanno lunghezze che differiscono al più di k . Scrivere una funzione $bal(T, k)$ che dato in input l'albero T e un valore k verifica se T è k -bilanciato e ritorna un corrispondente valore booleano. Valutarne la complessità.



$bal(T, k) \rightarrow$ es. $k = 1$ come da esempio

```

x = T.root
if (x == nil)
    return true
else
    // depth (left, right) ≤ k
    left_depth = bal(x.left, k)
    right_depth = bal(x.right, k)
    if (left_depth - right_depth ≤ k)
        return true
    else
        return false

```

```

bal(T,k) // verifica se l'albero radicato in x e' k-bilanciato,
// ovvero i cammini dalla radice terminati da nil hanno
// lunghezze che differiscono al piu di k

```

```

// si basa su di una funzione ricorsiva che calcola la
// lunghezza minima e massima dei cammini nil-terminated
// da un certo nodo.

```

```

min,max = nilTerm(T.root)

```

```

return max-min ≤ k

```

```

nilTerm(x)

```

```

if x = nil
    return (0,0)

```

```

else

```

```

(left_min, left_max) = nilTerm(x.left)

```

```

(right_min, right_max) = nilTerm(x.right)

```

```

return min(left_min, right_min) + 1, max(left_max, right_max) + 1

```

Vogliamo verificare se l'albero è k -bilanciato (quindi, i cammini dalle radici alle foglie differiscono al max di k) $\rightarrow$ variante dell'esercizio 1-bilanciato. Quindi, ricorriamo su tutti i nodi e ritorniamo due variabili (min-max) la cui differenza

permette di intuire il bilanciamento degli alberi. Infatti:

- ricorriamo a sinistra e salviamo il nodo più a sx e il nodo più a dx (min e max) del sottoalbero sx

- ricorriamo a destra e salviamo il nodo più a dx e il nodo più a sx (min e max) del sottoalbero dx

Assieme al nodo chiave, sommiamo 1 al percorso a sx e al percorso a dx, così prendiamo chiave e tutti i nodi.

K - BOUNDED

Perché Servono Entrambi Min e Max

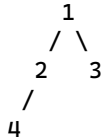
****Il problema con una singola variabile:****

Se calcoli solo la ****profondità massima**** (depth), perdi l'informazione sui cammini più corti. La definizione richiede:

```
```  
max_lunghezza - min_lunghezza ≤ k
```
```

Con un solo valore non puoi fare questo confronto.

****Controesempio:****



****Con solo depth (max):****

```
depth(4) = 1  
depth(2) = max(depth(4), depth(nil)) + 1 = max(1, 0) + 1 = 2  
depth(3) = 0 + 1 = 1  
depth(1) = max(2, 1) + 1 = 3
```

Ottieni `depth = 3`. Ma con $k=1$, è 1-bilanciato?

****Cammini reali:****

- 1→2→4→nil: lunghezza 3
- 1→3→nil: lunghezza 2
- Differenza: $3 - 2 = 1$ ✓ (è 1-bilanciato)

****Il problema:**** Se avessi solo `depth = 3`, come fai a sapere che esiste un cammino di lunghezza 2?
****Non puoi.**** Hai perso quell'informazione durante la ricorsione.

****Perché (min, max) risolve il problema:****

```
nilTerm(x):  
    if x = nil:  
        return (0, 0)  
    else:  
        (left_min, left_max) = nilTerm(x.left)  
        (right_min, right_max) = nilTerm(x.right)  
        return (min(left_min, right_min) + 1, max(left_max, right_max) + 1)
```

****Esecuzione:****

- `nilTerm(4) = (1, 1)`
- `nilTerm(3) = (1, 1)`
- `nilTerm(2) = (min(1,0)+1, max(1,0)+1) = (1, 2)` ← ****mantieni il minimo!****
- `nilTerm(1) = (min(1,1)+1, max(2,1)+1) = (2, 3)` ← ****hai entrambi****

Ora puoi verificare: $3 - 2 = 1 \leq k$ ✓

****Conclusione:****

Non è questione di "vantaggio", è ****necessità matematica****:

- La proprietà k -bilanciato richiede ****due**** valori: min e max
- Con un solo valore (depth o min da solo) non hai abbastanza informazione
- Ogni nodo deve propagare verso l'alto sia il cammino più corto che il più lungo del suo sottoalbero

****È comunque una singola run:**** $O(n)$ tempo, visiti ogni nodo una volta sola.
Ma ogni nodo restituisce una coppia invece di un singolo valore.

$BAL(T, x) \rightarrow IF (0, 0) \text{ return } NIL$

ELSE

$IF (min - max \leq k)$

$\rightarrow TRUE$

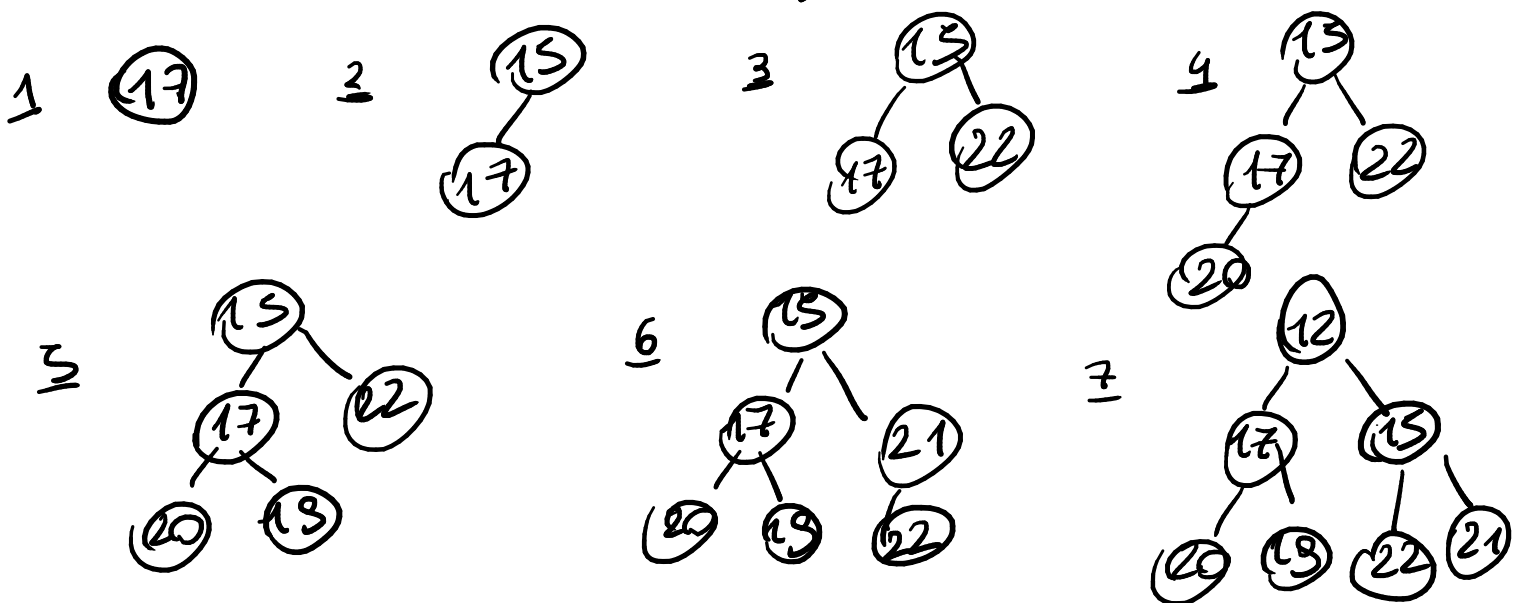
ELSE $\rightarrow FALSE$

Domanda B (7 punti) Dare la definizione di min-heap. Data la sequenza di elementi 17, 15, 22, 20, 19, 21, 12, si specifichi il min-heap ottenuto inserendo, a partire da uno heap vuoto, uno alla volta questi elementi nell'ordine indicato e infine rimuovendo 17. Si descriva sinteticamente come si procede per arrivare al risultato.

MIN-HEAP $\rightarrow$ ALBERO BINARIO QUASI COMPLETO

$$\begin{pmatrix} A[i] \leq A[2i] \\ A[i] \leq A[2i+1] \end{pmatrix}$$

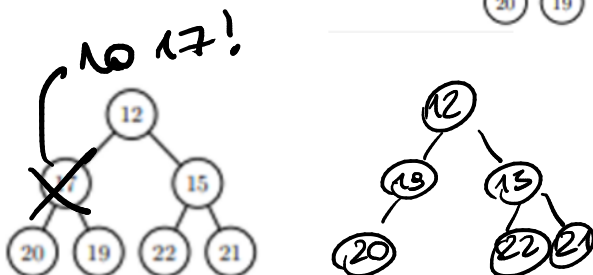
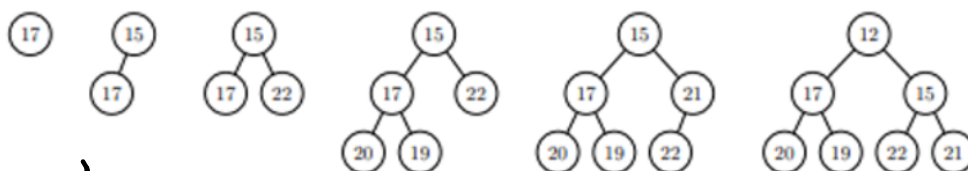
[17, 15, 22, 20, 19, 21, 12]



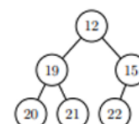
Soluzione: Un min-heap è un albero binario ordinato quasi-completo con la proprietà che per ogni nodo x , se x non è radice, il genitore di x ha chiave minore o uguale a quella di x , o equivalentemente ogni nodo x ha chiave minore o uguale a quella dei suoi successori.

Si procede inserendo nel min-heap i vari elementi con la procedura **HeapInsert** a partire da heap vuoto. La procedura inserisce l'elemento come prima foglia utile (ultimo elemento dell'array) e richiama la procedura **MinHeapifyUp** per ripristinare la proprietà di min-heap.

Gli inserimenti nell'ordine producono gli heap indicati in figura



Infine, la rimozione di un nodo con chiave x si realizza sostituendolo con l'ultima foglia y e richiama **MinHeapifyUp**, se $y < x$ oppure **MinHeapify**, se $y > x$. Nello specifico per eliminare 17 lo si rimpiazza con l'ultima foglia 21 e si richiama **MinHeapify**, ottenendo il min-heap



che in forma di array è [12, 19, 15, 20, 21, 22].

```

DELETE(A, i)
1  old = A[i]
2  A[i] = A[A.heapsize]
3  A.heapsize = A.heapsize - 1
4  if old ≤ A[i]
5      MAX-HEAPIFY-UP(A, i)
6  else
7      MAX-HEAPIFY(A, i)

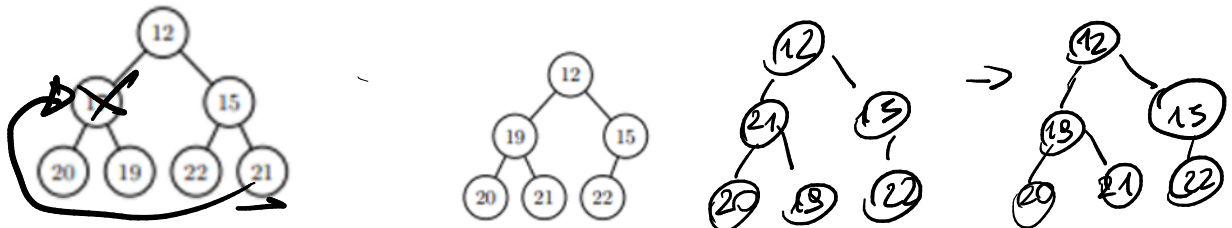
```

Cancellazione del Nodo 17

Algoritmo di cancellazione in un min-heap:

1. Sostituisci con l'ultima foglia: L'ultima foglia nell'ordine di riempimento è 21
2. Rimuovi l'ultima foglia
3. Applica MinHeapify verso il basso

Infine, la rimozione di un nodo con chiave x si realizza sostituendolo con l'ultima foglia y e richiamando MinHeapifyUp, se $y < x$ oppure MinHeapify, se $y > x$. Nello specifico per eliminare 17 lo si rimpiazza con l'ultima foglia 21 e si richiama MinHeapify, ottenendo il min-heap



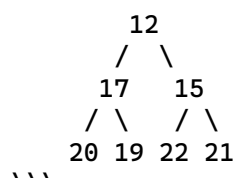
che in forma di array è [12, 19, 15, 20, 21, 22].

```

DELETE(A, i):
1  old = A[i]
2  A[i] = A[A.heapsize]
3  A.heapsize = A.heapsize - 1
4  if old ≥ A[i]                                // CAMBIATO: ≥ invece di ≤
5      MIN-HEAPIFY-UP(A, i)                     // CAMBIATO: MIN invece di MAX
6  else
7      MIN-HEAPIFY(A, i)                       // CAMBIATO: MIN invece di MAX

```

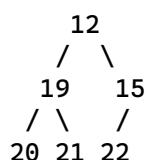
****Heap originale:****



****Esecuzione:****

1. 'old = 17'
2. 'A[2] = A[7] = 21' → sostituisci 17 con l'ultima foglia 21
3. 'heapsize = 6'
4. 'if 17 ≥ 21' → ****FALSE****
5. Salta MIN-HEAPIFY-UP
6. ****Esegui MIN-HEAPIFY(A, 2)**** verso il basso:
 - Figli di 21: 20 (sinistra), 19 (destra)
 - 'min(21, 20, 19) = 19'
 - Scambia 21 con 19

****Risultato finale:****



Domanda A (7 punti) Dare la definizione della classe $\Theta(f(n))$. Mostrare che la ricorrenza

$$T(n) = \frac{1}{4}T(n-1) + 3n^2$$

ha soluzione in $\Theta(n^2)$.

Dimostrare le due disuguaglianze ($\leq cn^2$ e $\geq dn^2$) tali che valga esattamente la condizione theta

$$T(n) = \frac{1}{4}T(n-1) + 3n^2 \quad \sim \quad \Theta(n^2)$$

$$T(n^2) \leq C(n^2) \quad \rightarrow \quad \frac{1}{4}T(n-1)^2 + 3n^2 \leq Cn^2$$

$$\rightarrow \frac{1}{4}C(n+1)^2 + 3n^2 \leq Cn^2$$

$$(C + 2Cn > 0) \quad \rightarrow \quad \frac{1}{4}Cn^2 + C + 2Cn + 3n^2 \leq Cn^2$$

$$\rightarrow \frac{1}{4}Cn^2 + 3n^2 \leq Cn^2$$

$$\rightarrow 3n^2 \leq Cn^2 - \frac{1}{4}Cn^2$$

$$\rightarrow 12n^2 \leq \frac{4Cn^2 - Cn^2}{4} \cdot 4$$

$$\rightarrow 12n^2 \leq 3Cn^2$$

$$\rightarrow C \geq 4$$

$$T(n^2) \geq d(n^2)$$

$$\frac{1}{4}d(n-1)^2 + 3n^2 \geq dn^2 \quad \dots \quad \rightarrow d \geq 3$$

12 ques. LA cui somma è 1

$$|S_j| \leq 2 \quad \text{e} \quad \sum_{a \in S_j} a \leq 1, \quad 1 \leq j \leq k.$$

→ 1. Scrivere il codice di un algoritmo greedy che restituisce un (2,1)-boxing ottimo in tempo lineare. (Suggerimento: si creino i sottoinsiemi in modo opportuno basandosi sulla sequenza ordinata.)

2. Si enunci la proprietà di scelta greedy per l'algoritmo sviluppato al punto precedente e la si dimostri, cioè si dimostri che esiste sempre una soluzione ottima che contiene la scelta greedy.

soluzione ottima che contiene la scelta greedy.

[FIRST] [LAST]

0.1 0.2 + 0.8 1.0

 ↗ ↘ ↖

 1

```

for i = 2 to n
    if (S[i] + OPT ≤ 1)
        OPT = OPT U {S_first, S_last})
        first = first + 1
    else
        OPT = OPT U {S_last}
        last = last - 1
return OPT

```

$$\frac{0.1}{1} + \frac{0.9}{1} \leq 1$$

2. La scelta greedy è $\{a_1, a_n\}$ se $n > 1$ e $a_1 + a_n \leq 1$, altrimenti $\{a_n\}$. Ora dimostriamo che esiste sempre una soluzione ottima che contiene la scelta greedy. I casi $n = 1$ e $a_1 + a_n > 1$ sono banali, visto che in questi casi ogni soluzione ammissibile deve contenere il sottoinsieme $\{a_n\}$. Quindi assumiamo che la scelta greedy sia $\{a_1, a_n\}$. Consideriamo una qualsiasi soluzione ottima dove a_1 e a_n non sono accoppiati nello stesso sottoinsieme. Quindi, esistono due sottoinsiemi S_1 e S_2 , con $a_1 \in S_1$ e $a_n \in S_2$. Sostituiamo questi due sottoinsiemi con $S'_1 = \{a_1, a_n\}$ (cioè, la scelta greedy) e $S'_2 = S_1 \cup S_2 \setminus \{a_1, a_n\}$. $|S'_2| \leq 2$ e, se $|S'_2| = 2$, allora $S'_2 = \{a_s, a_t\}$ con $a_s \in S_1$ e $a_t \in S_2$. Siccome a_t era precedentemente accoppiato con a_n , a maggior ragione può essere accoppiato con $a_s < a_n$, quindi la nuova soluzione così creata è ammissibile e ancora ottima.

Esercizio 2 (8 punti) Per $n > 0$, siano dati due vettori a componenti intere $\mathbf{a}, \mathbf{b} \in \mathbb{Z}^n$. Si consideri la quantità $c(i, j)$, con $0 \leq i \leq j \leq n-1$, definita come segue:

$$c(i, j) = \begin{cases} a_i & \text{se } 0 < i \leq n-1 \text{ e } j = n-1, \\ b_j & \text{se } i = 0 \text{ e } 0 \leq j \leq n-1, \\ c(i-1, j) \cdot c(i, j+1) & 0 < i \leq j < n-1. \end{cases}$$

Si vuole calcolare la quantità $M = \max\{c(i, j) : 0 \leq i \leq j \leq n-1\}$.

$M(n, 1)$

1. Si fornisca il codice di un algoritmo iterativo bottom-up per il calcolo di M .
2. Si valuti la complessità esatta dell'algoritmo, associando costo unitario ai prodotti tra numeri interi e costo nullo a tutte le altre operazioni.

COMPUTE(a, b)

$n = \text{length}(\mathbf{A})$

// assumiamo n uguale per a e per b

$M = -\text{INFITY}$

for $i = 1$ to $n-1$

$C[i, n-1] = a[i]$

$M = \text{MAX}(M, C[i, n-1])$

for $j = 0$ to $n-1$

$C[0, j] = b[j]$

$M = \text{MAX}(M, C[0, j])$

for $i = 1$ to $n-2$

for $j = n-2$ downto i

$C[i, j] = C[i-1, j] \cdot C[i, j+1]$

$M = \text{MAX}(M, C[i, j])$

return $M[N, 1]$

COMPLESSITÀ?

$$\sum_{i=1}^{n-2} \sum_{j=i}^{n-2} 1 = \sum_{i=1}^{n-2} (n-2-(i-1))$$

$$\sum_{i=1}^{n-2} (n-1-i) = \sum_{k=1}^{n-2} k$$

$$\frac{(n-2)(n-1)}{2}$$